Software Engineering

CSC648/848

Henry
VILLAR

ARCHITECTURE INTRO & SE TEAMWORK

Intro to Architecture Design Pattern & Teamwork vs. group work,

Week 2 Summary

1. What are Personas?
2. What are Use Cases?
3. What are Requirements?
4. What are Specifications?
5. What are Priorities, how do you prioritize?
6. Mockups and Storyboards?

Summary. Cont.

1. What are Personas?

   Character that represents the different user types that might use your applications (service, product, site, or brand in a similar way)

2. What are Use Cases?

   Services and functions personas will use your envisioned system

3. What are Requirements?

   Abstract statements or descriptions of SW services, functions or constraints focus on WHAT

   - *Functional (what functions the SW has, what does it do)*
   - *Non-functional – what properties the SW has (security, privacy, performance, deploy in Cloud? etc.)*

4. What are Specifications?

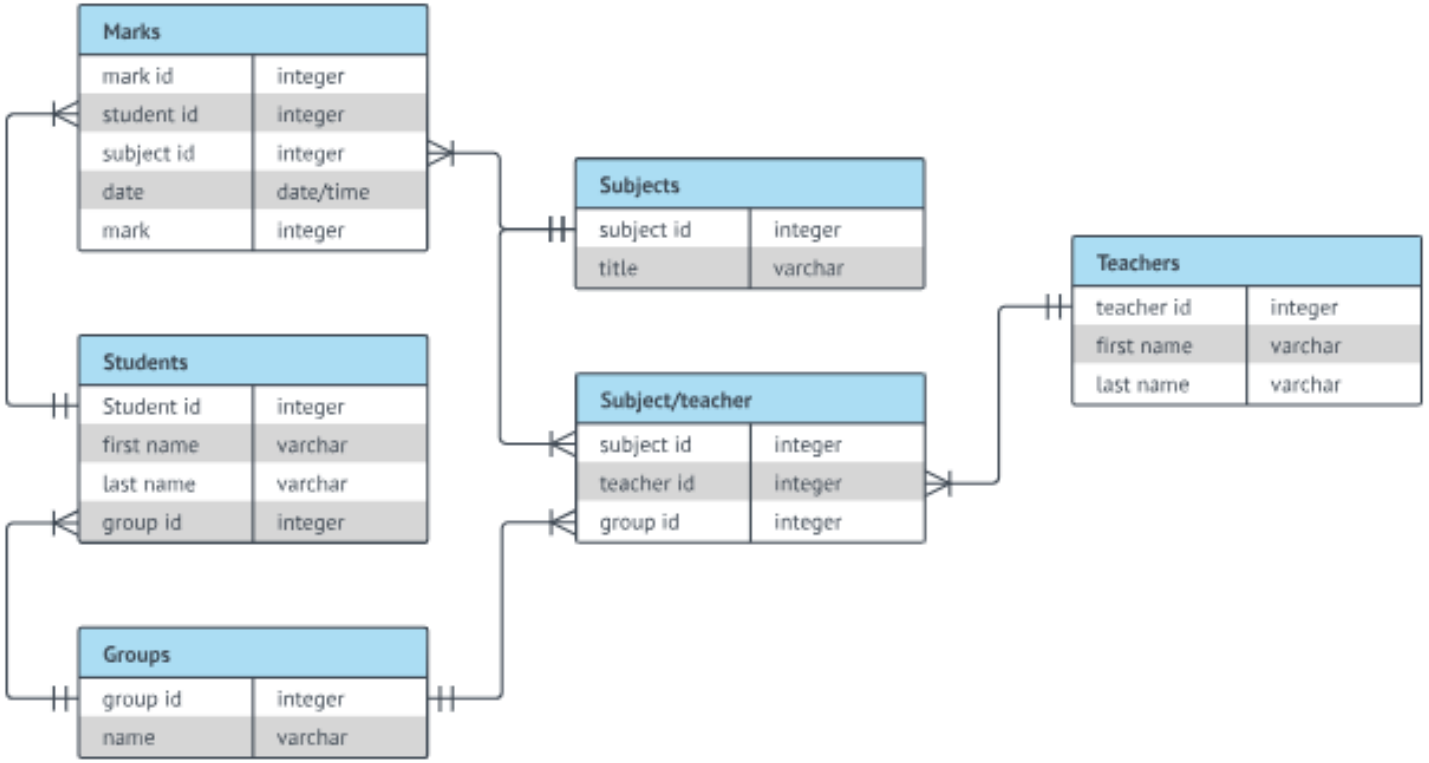
   More concrete engineering guidance of HOW to develop the SW

5. What are Priorities, how do you prioritize?

   It is critical to have priorities attached to each requirement so you can make choices when schedules and costs and resources get constrained. P1 Critical (must have), P2 Important (should/Could have), P3 Opportunistic (Nice to have)

6. What are Mockups and Storyboards?

   Simplified graphical representations of envisioned UI to guide the implementation and the development understanding



Data Items and Entities (Entity Relationship Diagram – ERD)

Example requirements for the insulin pump software system

3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. (*Changes in blood sugar are relatively slow, so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.*)

3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. (*A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.*)

Copyright ©2016 Pearson Education, All Rights Reserved

Source: Medical Economics
https://www.medicaleconomics.com/view/certain-insulin-pumps-risk-hacking

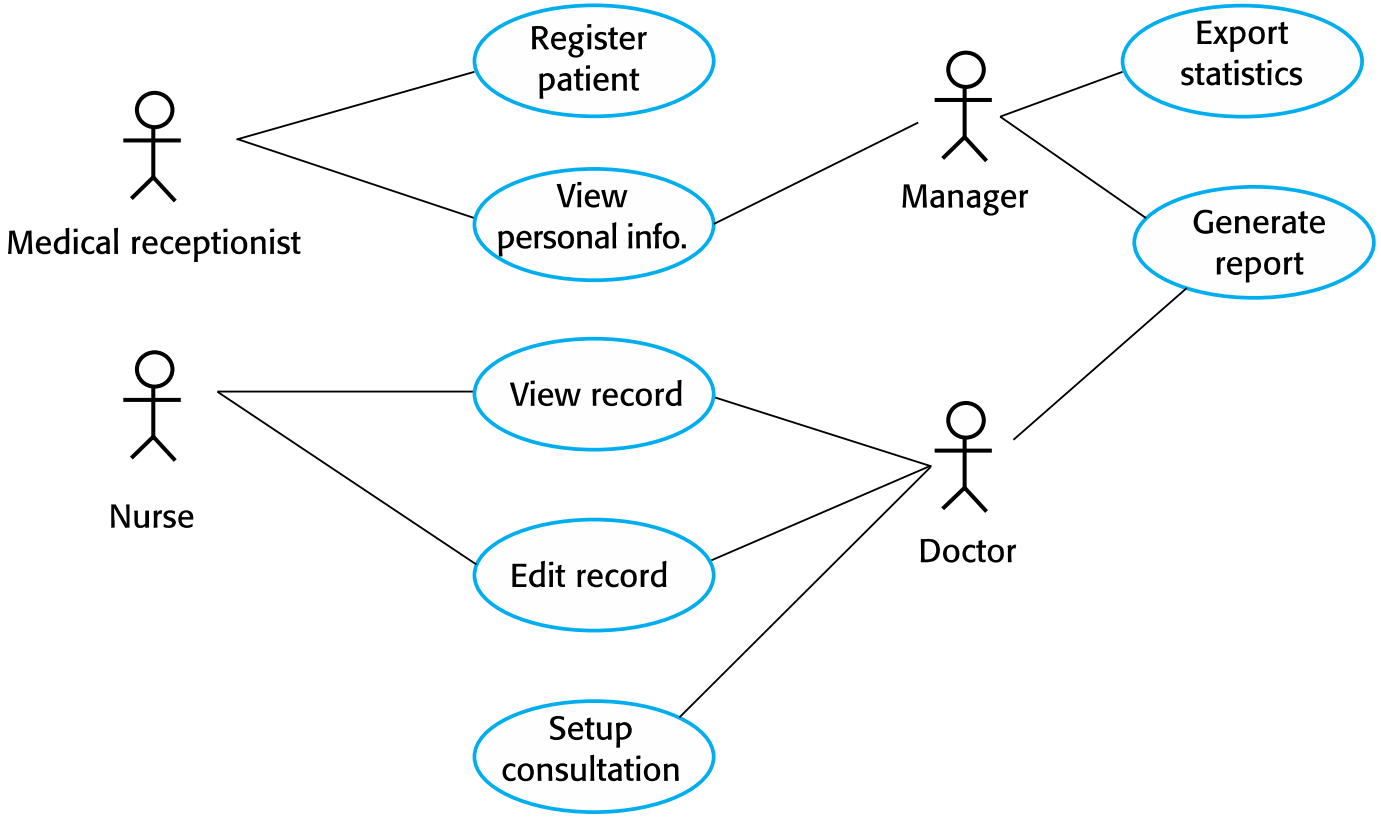
The structured specification of a requirement for an insulin pump

Insulin Pump/Control Software/SRS/3.3.2

Function	Compute insulin dose: Safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2), the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose—the dose in insulin to be delivered.
Destination	Main control loop.
Action:	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered. (see Figure 4.14)
Requires	Two previous readings so that the rate of change of sugar level can be computed.
Precondition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Postcondition	r0 is replaced by r1 then r1 is replaced by r2.
Side effects	None.

Copyright ©2016 Pearson Education, All Rights Reserved

Use cases for the Mentcare system



W2 Applicability – Class Exercise

- SCENARIO

Your start-up is assigned to develop a web application for project participation metrics reporting service for instructors as a collaborative platform for teaching, training and managing teamwork in academia or industry. This is very important application for educators and software managers since teamwork during software development is increasingly becoming one of the key skills required for students to be competitive in a global development environment.

Users of the system are instructors teaching collaborative project development, TA, Software Managers, Product Managers, Students and Developers Engineers. Some instructors' level of computing knowledge should be considered low but they are familiar with using basic Web and office applications (word processing etc.). They do not necessarily understand internals of Web and database applications. They will not be trained in using this system; hence they expect considerable ease of use and an intuitive interface. But most of the other users are computer savvy.

The app needs to collect the following data: Version Repository, Bug Tracking, Mailing List, Discussion Forum, Documentation among other.

The app needs to display the following information as minimum: Tabular and Graphical Data comparing all current projects

Users shall have to register and then login in order to use the system

System shall respond to user query in less than 3 seconds.

On-line help shall be provided

Cont.

- ASK
  - PENDING. Executive Summary
  1. Personas & Use Cases
  2. Data items and entities
  3. Functional Requirements
  4. Non-functional requirements
  5. Mockup and Storyboard
  6. Competitive Analysis
  - PENDING. High-Level system architecture
  - Team & Roles

W2 Applicability – Class Exercise

- SCENARIO

Your start-up is assigned to develop a web application for project participation metrics reporting service for instructors as a collaborative platform for teaching, training and managing teamwork in academia or industry. This is very important application for educators and software managers since teamwork during software development is increasingly becoming one of the key skills required for students to be competitive in a global development environment.

Users of the system are instructors teaching collaborative project development, TA, Software Managers, Product Managers, Students and Developers Engineers. Some instructors' level of computing knowledge should be considered low but they are familiar with using basic Web and office applications (word processing etc.). They do not necessarily understand internals of Web and database applications. They will not be trained in using this system; hence they expect considerable ease of use and an intuitive interface. But most of the other users are computer savvy.

The app needs to collect the following data: Version Repository, Bug Tracking, Mailing List, Discussion Forum, Documentation among other.

The app needs to display the following information as minimum: Tabular and Graphical Data comparing all current projects

Users shall have to register and then login in order to use the system

System shall respond to user query in less than 3 seconds.

On-line help shall be provided

Requirements

Example Product: Student Assignment Submission System

Requirements

- R1. User Login
  The system shall allow students to log in securely using a username and password.
- R2. Assignment Upload
  The system shall allow students to upload assignment files in PDF or DOCX format.
- R3. Submission Deadline Enforcement
  The system shall prevent submissions after the deadline.
- R4. Instructor View
  The system shall allow instructors to view and download submitted assignments.
- R5. Data Storage
  The system shall store all submissions securely in the cloud.

Engineering Actions Based on Requirements

For R1 – User Login

- Design a login UI page
- Implement authentication using hashed passwords
- Connect login system to a user database
- Add session management for logged-in users
- Test login success/failure scenarios

For R2 – Assignment Upload

- Create file upload interface
- Add file type validation (PDF/DOCX only)
- Implement backend upload API
- Store files in cloud storage
- Add error handling for failed uploads

Today’s Topics

- Intro to SW Architecture and Design Patterns
- SE Teamwork

Class Activity: Canvas/Architect Role

- Mockup of an iWatch UI
- Architecture of iWatch (All Components)

Class Activity

- Architect / Designer Scenario
- Scenario: **Apple Watch Application for an AI tool interacting with any existing Health application**
  - Front End Design (UI)
  - Architecture Design Pattern
- Duration: 5 min
- Deliverables (via text, block diagrams, or UML):
  - Apple watch Mock-up of the AI tool UI
  - Chosen Architecture Design Pattern

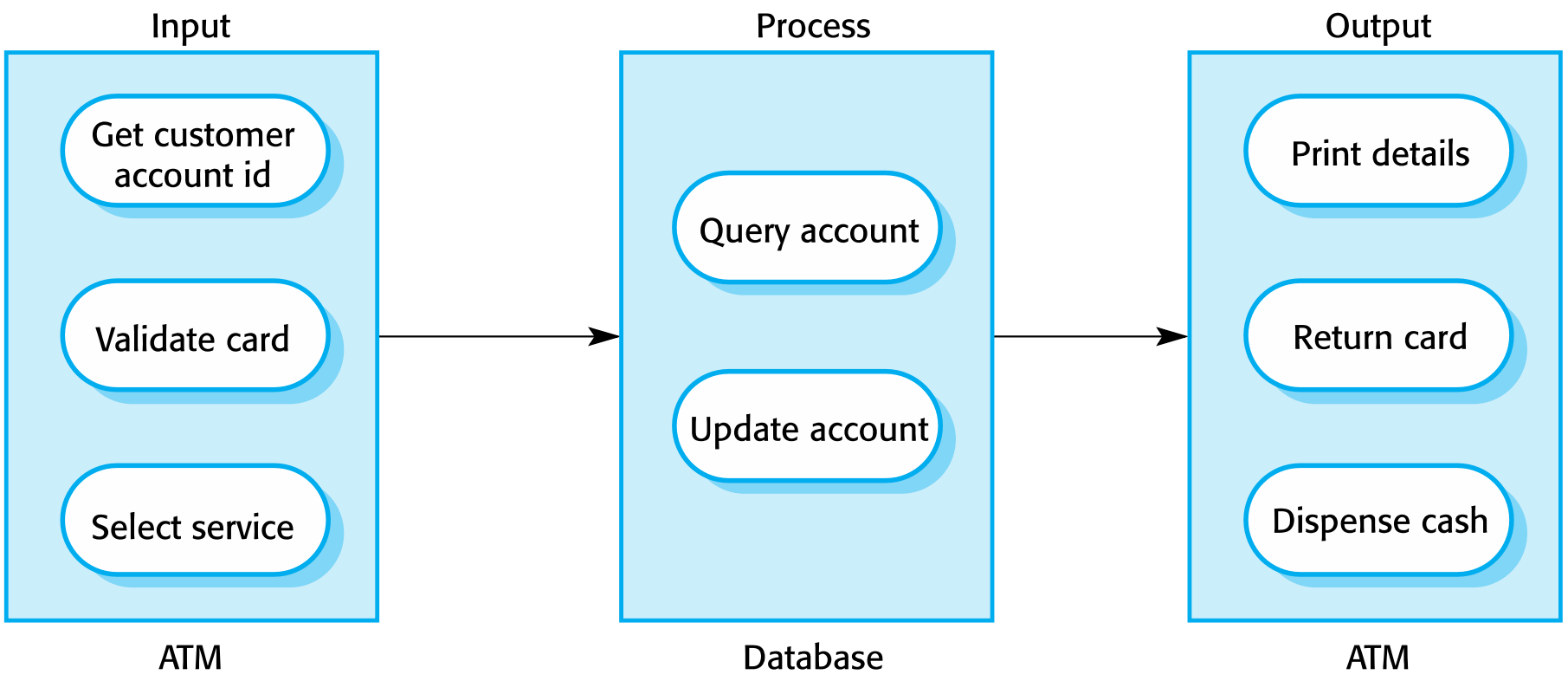
SW Architecture and Design Patterns

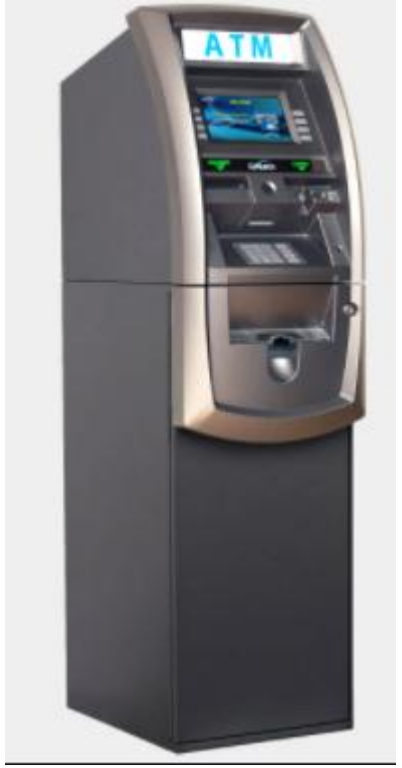
- Understand the importance and role of SW architecture design
- Arch. Design patterns: usage and benefits
- UML – Unified Modeling Language

Architecture Design

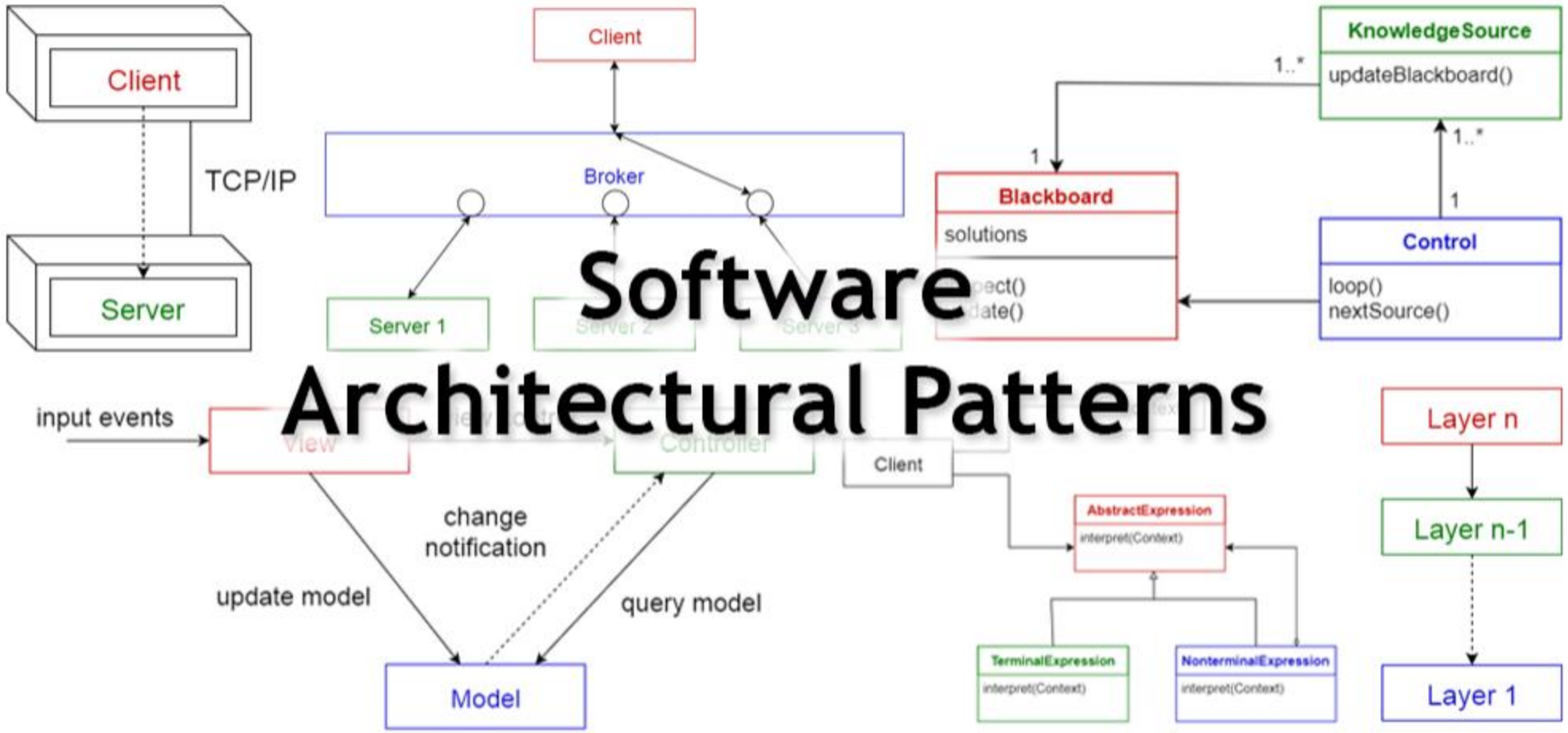
- **Architectural design** is concerned with understanding how a software system should be organized and designing the overall structure of that system.
- **Architectural design** is the critical link between design and requirements engineering, as it identifies the main structural <u>components</u> in a system and the relationships between them.
- **The output** of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components.

The software architecture of an ATM system





Source: https://empireatmgroup.com/shop/gen mega-2500-atm-machine/



Software Architectural Patterns

Source: Internet

Important

Architectural design patterns

- Predefined and proven *high level* solutions for particular class of problems
- Usage: *Analyze* your application requirements then try to *map* into one of the known architectural patterns, then evaluate to chose the best
- Examples:
  - Model-View-Controller;
  - Multitier architectures. Layered Architecture (or Three (Four) Tier Arch.);
  - Repository;
  - Client-Server
- http://en.wikipedia.org/wiki/Architectural_pattern

Some resources on architectural patterns

- MVC
  - http://en.wikipedia.org/wiki/MVC_Pattern
- Multitier architectures
  - http://en.wikipedia.org/wiki/3-tier_architecture
- Client server
  - http://en.wikipedia.org/wiki/Client_server
- Blackboard
  - http://en.wikipedia.org/wiki/Blackboard_system

MVC – preferred model/pattern for the team project

- Good explanation with example codes
  - http://www.tomdalling.com/blog/software-design/model-view-controller-explained/

- What each subsystem "needs to know" about others? (The less it knows the more modular it is…)
  - There are some differing definitions as to what model can do and knows about – does it also do some business logic/data interpretation?
  - We prefer that model simply manages data, and business logic is separate (in controller)
    - Why is this good?

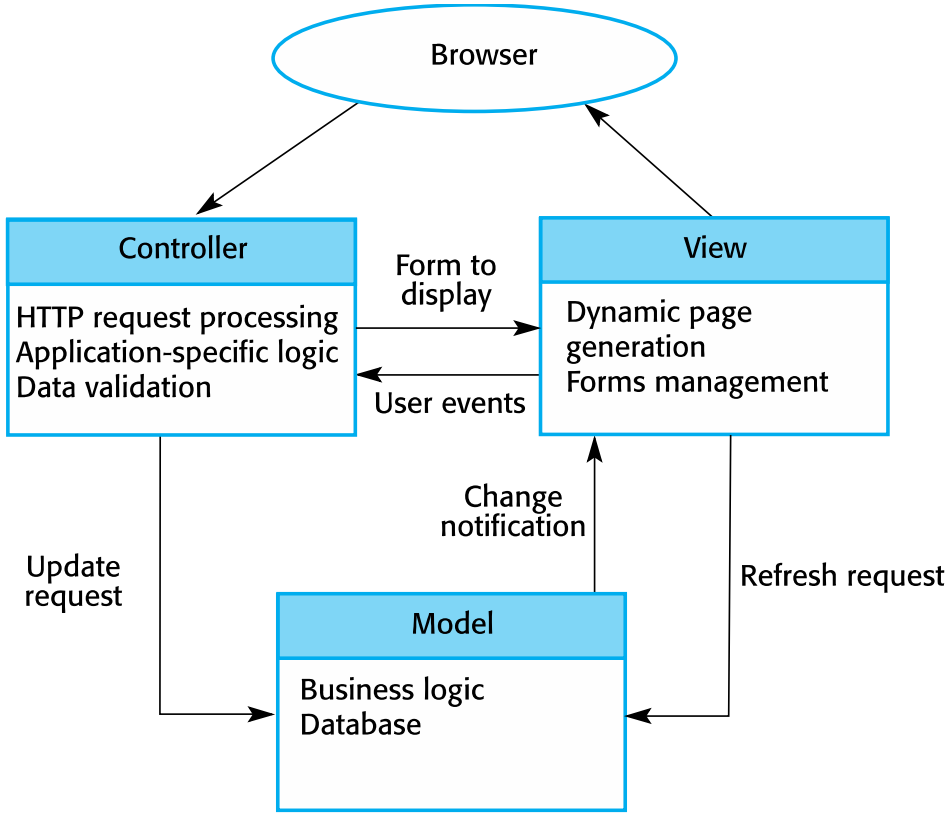
The Model-View-Controller (MVC) pattern

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model. See Figure 6.3.
Example	Figure 6.4 shows the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them.
Disadvantages	Can involve additional code and code complexity when the data model and interactions are simple.

Important

Model-View-Controller (in the context of Internet) (adapted from I. Sommerwille)

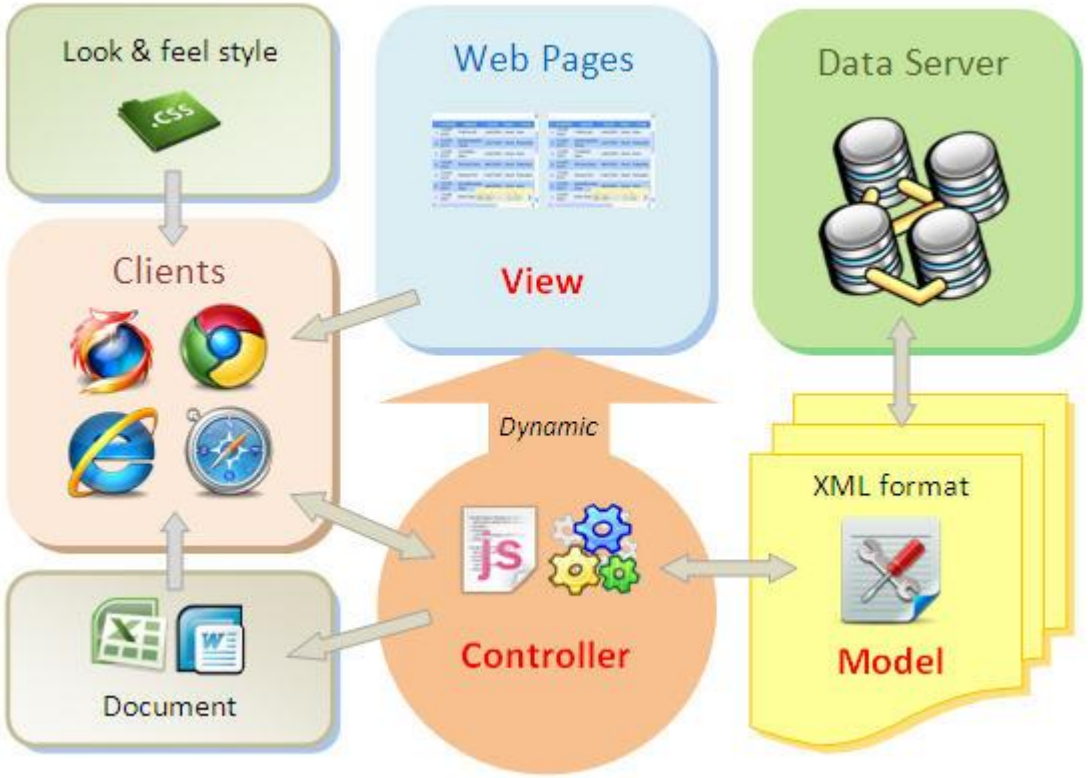
Advantages:
Independently treats Data from presentation;
Easy to change data presentation

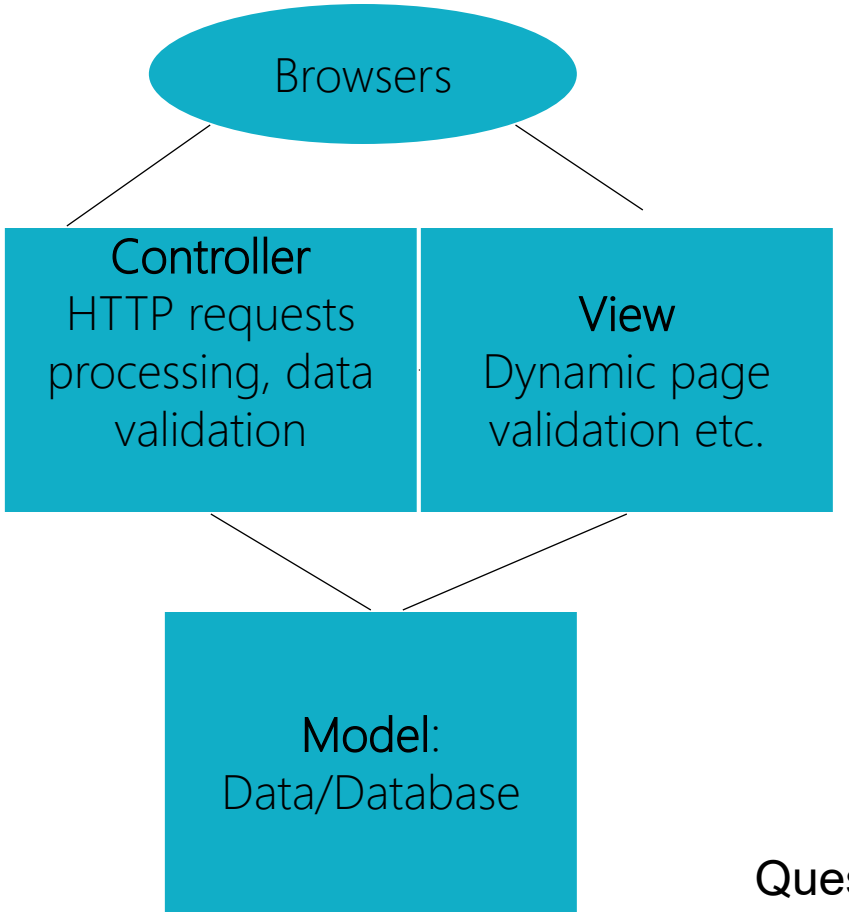


Disadvantages:
Adds complexity

Important

Model-View-Controller modification





Question:
where does search code belong?

Source: https://webformdeveloping.wordpress.com/2014/04/07/mvc/

The Layered architecture pattern

Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system. See Figure 6.6.
Example	A layered model of a system for sharing copyright documents held in different libraries, as shown in Figure 6.7.
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

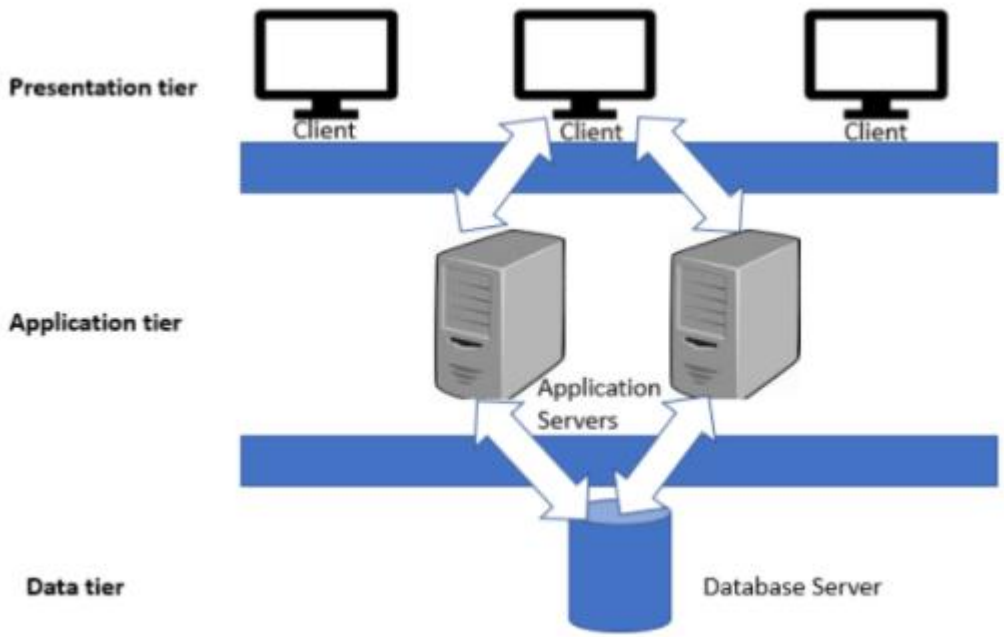
A generic layered architecture

User interface

User interface management
Authentication and authorization

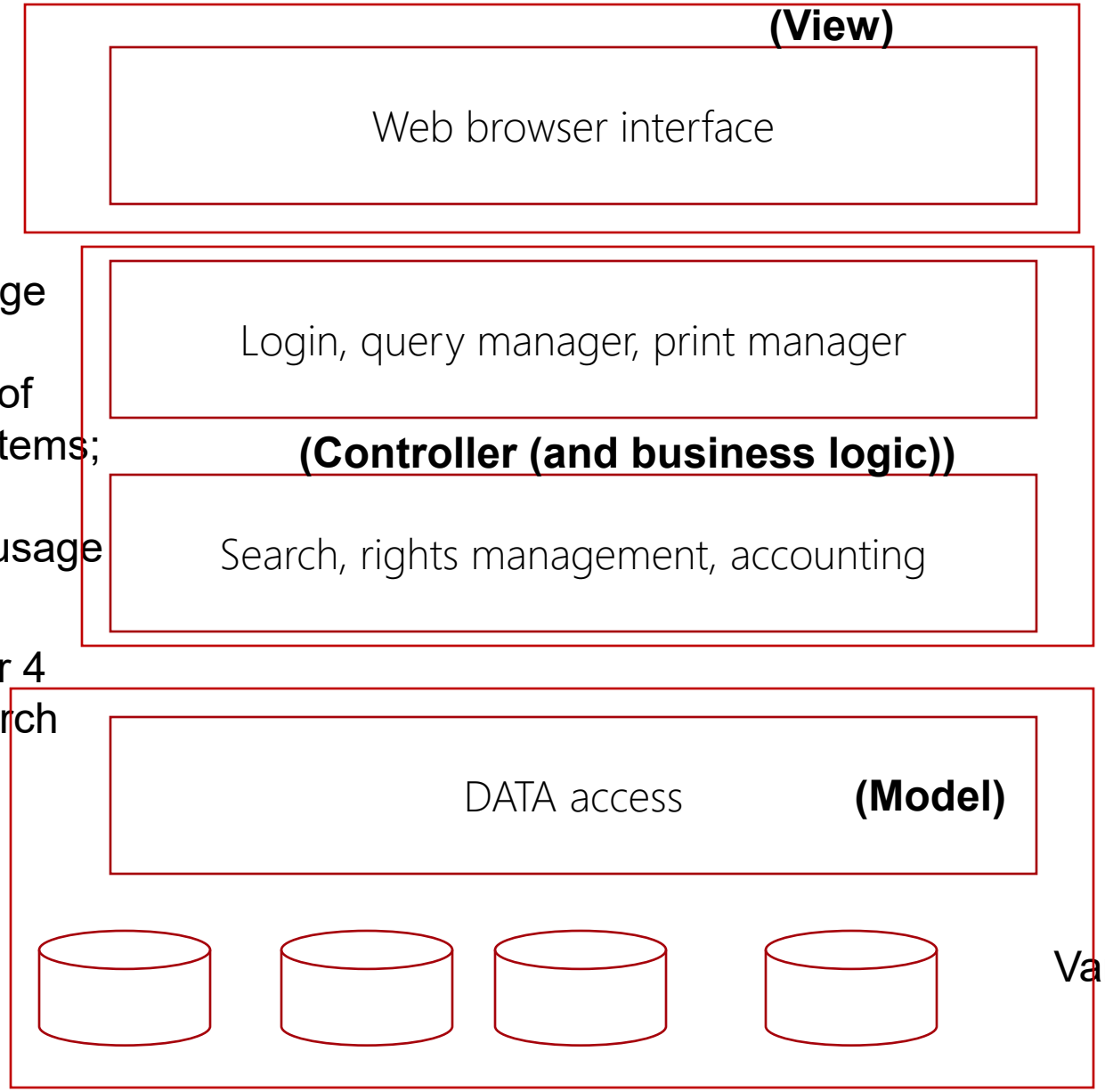
Core business logic/application functionality
System utilities

System support (OS, database etc.)



Source: https://hub.packtpub.com/what-is-multi-layered-software-architecture/

Layered Architecture: example of Internet based library system (adapted from I. Sommerwille)



Important

Usage:
Complex large systems;
Built on top of Existing systems;
Security;
Formal DB usage

Variant: 3 or 4 tier WWW arch

Advantage
Modular;
Easy to interface and replace layers

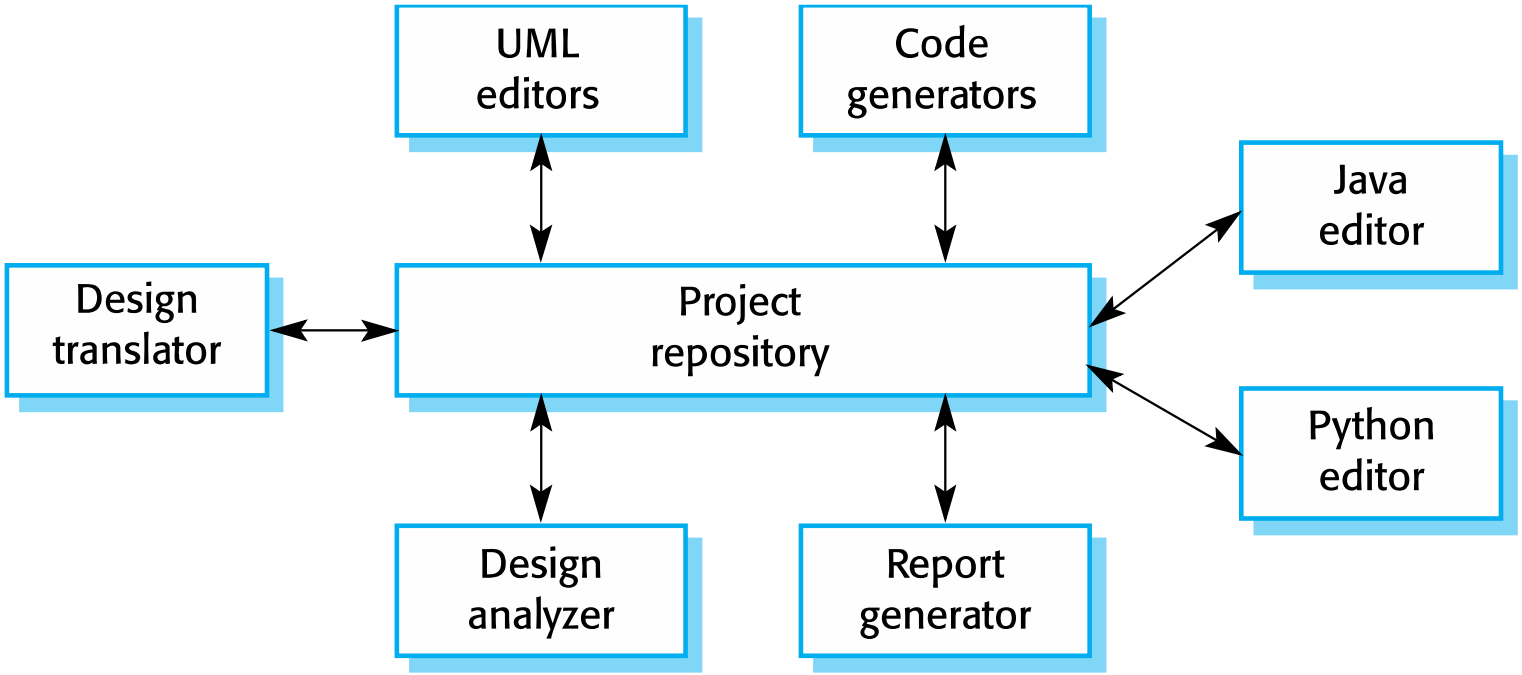
Disadvantage
Complex;
Not always easy To separate layers

Various databases

The Repository pattern

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Figure 6.9 is an example of an IDE where the components use a repository of system design information. Each software tool generates information which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent—they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

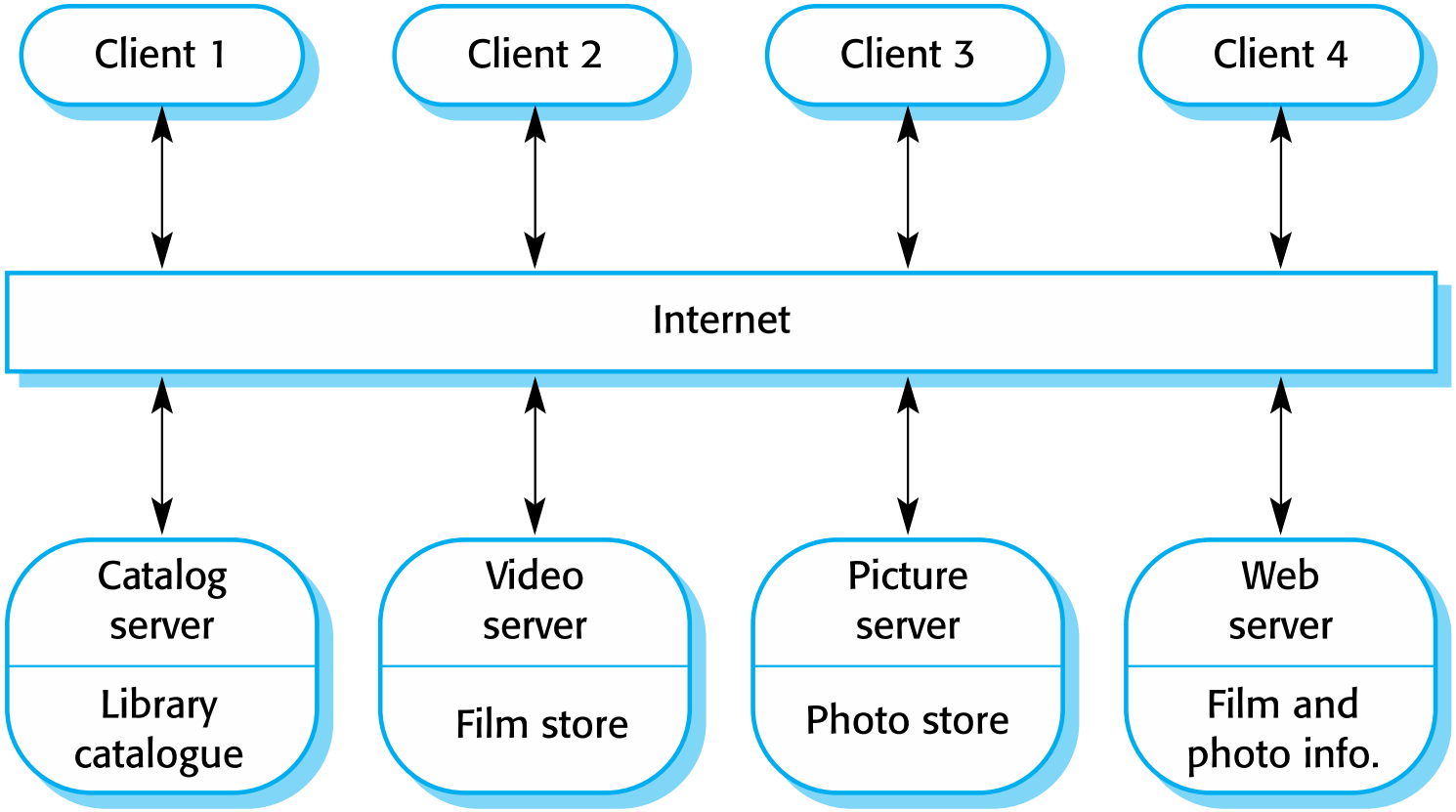
A repository architecture for an IDE



The Client–server pattern

Name	Client-server
Description	In a client–server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
Example	Figure 6.11 is an example of a film and video/DVD library organized as a client–server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.

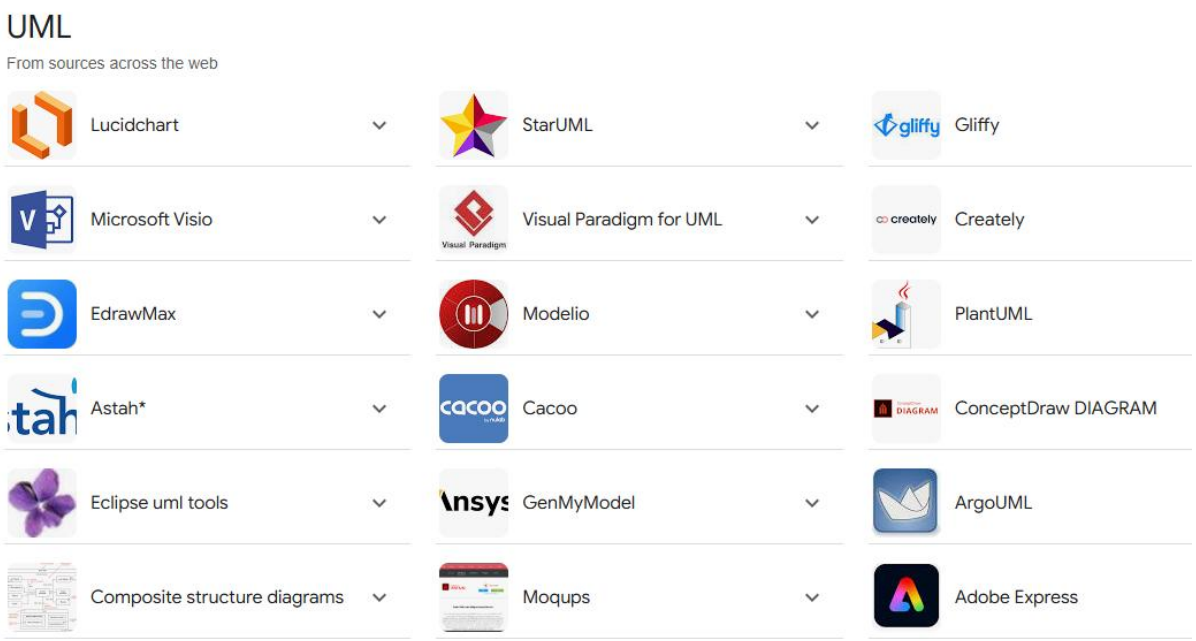
A client–server architecture for a film library



Important

How to communicate/describe high level architecture

- Text (list components)
- Simple block diagrams (informal) – sometimes with special tools
- Unified Modeling Language – UML – formal way
  - http://edn.embarcadero.com/article/31863
  - UML Tools: https://en.wikipedia.org/wiki/List_of_Unified_Modeling_Language_tools



SE TEAMWORK

SFSU CS Objectives:

Students will be able to solve problems working **in group settings**

Important

Teamwork - General

- Teamwork vs. group work
  - *Group work*: Working together to deliver M0 is group work – you help each other but each of you has to do his/her same M0 deliverable
  - *Teamwork*: Working on final class project is teamwork – you all work together on pieces to complete ONE deliverable

- Teamwork vs. group work
  - https://3back.com/well-formed-teams/teamwork-vs-group-work/
  - https://smallbusiness.chron.com/differences-between-group-work-team-work-11004.html

Essence of Teamwork

- From
  - https://3back.com/well-formed-teams/teamwork-vs-group-work/

- Sublimation of individual **ego** and need for recognition to the needs of the Team
- Sharing and **balancing workload** dynamically at daily or finer granularity
- Offering and requesting help in an environment of **trust**
- Willingness to make **personal sacrifices** in support of the Team
- Recognition that the **Team's goal outweighs** (but does not exclude) individual goals

Important

Current research on team success

- Team **success correlates with team cohesion** and NOT with the sum of team members IQs or expertise

IMPORTANT

Teamwork in SE

CRITICAL AND KEY FACTOR

Important

SW Management and teamwork

- **Process** (learned in this course)
- **Technology** (you studied it)

BUT also

- **People (surprise, this might be the key!)**

Two great reads

- Most SW project failures related to poor SW teamwork, management, process, not to technical issues
  - Charette, R. N: "Why Software Fails." *IEEE Spectrum*, September 2005, pp. 42.
- What Google leaned from its quest to build the perfect team – Google (NY Times 02/28/16)
  - http://www.nytimes.com/2016/02/28/magazine/what-google-learned-from-its-quest-to-build-the-perfect-team.html?_r=0
  - https://www.cnbc.com/2019/02/28/what-google-learned-in-its-quest-to-build-the-perfect-team.html

Important

What is SW management

- Project management
  - Organizing, planning and scheduling SW projects
- People management People Management
  - Hiring, motivating, organizing, rewarding, assigning tasks
- Cost Management
  - Planning and estimating costs, productivity, monitoring costs etc.
- Quality Management
  - Quality assurance, standards, planning, control, SW measurements and metrics
- Process management and improvement
  - Process measurement and assessment, analysis and modeling, SEI Capability Maturity Model (CMM)

Important

People "stuff" in SE

- People "stuff" is critical for success of SW projects
- People are the key and biggest asset in SW companies ➔ proper management and motivation of the **people** is the key
- Upper management, client management and users are **people**, and they have to be "managed" also
- People management involves some policies and rules, but above all it is about proper communication, motivation, listening and respect
- Balancing individualism v.s. team work
- SW Engineers productivity varies widely ➔ need to hire and motivate the best you can get

One of SFSU CS department's learning objectives –

http://cs.sfsu.edu/prospective-students/mission-objectives

Objective 3

- Students will be able to solve problems working **in group settings**. This translates to the following outcomes; students will demonstrate:
  - 3.1 Knowledge of basic SW engineering methods and practices, and their appropriate application
  - 3.2 Knowledge and application of collaborative tools for SW development
  - 3.3 Successful implementation of teamwork behavior and policies in a large class project

Important

SW development as a teamwork

- SW teams can range from 2 to several hundred
- Work together (same room) to different countries, language, time zone
- Best group size up to 8 members, responsible for a product or well defined subsystem
- Smaller groups are much better in terms of organization and communication ➔ critical factor
- Most innovation comes from 1-3 people, then gets implemented by larger teams

Important

SW development as a teamwork (2)

- Making groups work *effectively* is critical task of team leads and project managers
  - Development and communication of shared project vision and goals
  - Hiring
  - Getting team organized and trained
  - Motivating and sustaining motivation
  - Tracking progress
  - Providing adequate resources
  - Resolving conflicts (technical, personal, resource, schedule, priorities, comm. between SW team and stakeholders…)
  - Managing different phases differently (design, innovation, execution…)

Important

SW teams

- *Composition:* balance of skills, experience and personalities
- *Cohesiveness:* is this a team or collection of individuals
- *Communication:* do group members communicate effectively with each other
- *Organization:* is the organization such that everybody is effective and the whole team is productive? Can the best idea win? Can the team execute and deliver?

Managing conflict - some rules

Important

Always present, the key is how you manage it:

1. First try to resolve within the group among involved parties
2. If that does not work, involve team lead or trusted outside person
3. If that does not work involve management

For *global* environment:

1. First try to resolve issue within local groups
2. If that does not work involve team leads from all groups
3. If that does not work, escalate to management from all groups
4. Ultimately, one single person resolves the conflict

Team Project Time

- Work on M1
  - 7-10 pages. Email me direct link to PDF. Follow instructions
  1. Exec Summary
  2. Personas and main Use cases
  3. Data items and entities
  4. Functional Req
  5. Non-functional
  6. Competitive Analysis
  7. HL system architecture
  8. Team and roles
  9. Checklist

Study Materials

Processes/phases in teamwork (general) – http://en.wikipedia.org/wiki/Teamwork

- *Transition process* (between periods of action)
  - Mission analysis
  - Goal specification
  - Strategy formulation
- *Action process* (when the team attempts to accomplish its goals and objectives)
  - Monitoring progress toward goals
  - Systems monitoring
  - Team monitoring and backup behavior
  - Coordination
- *Interpersonal process* (present in both action periods and transition periods)
  - Conflict management
  - Motivation and confidence building
  - Affect management

Important

Professionals in terms of the motivation (Bass, Dunteman 1963)

- *Task oriented:* motivated by the work they do. Often like to work alone
- *Self-oriented:* primarily interested by personal success. Often like to work alone
- *Interaction-oriented:* motivated by interacting with co-workers and users (important for UI). Usually better communicators, like to work in a team

- What are you?
- As a manager, know this for your people and assign them to roles consistent with their motivation

Important

SW team organization

- *Informal, with implicit technical leader*. No formal authority exists. Decisions made democratically. Works best for small groups
- *Extreme programming* concept is the variant of the above where programmers work in a team of two, and all decisions are made in a group setting. Requires highly competent and motivated people
- *Chief programmer team:* very skilled person responsible for all key decisions and implementation, the rest are supporting him/her. May work, but may cause resentment, and requires really good chief programmer
- *Project manager, team lead, team members*: most common today

Important

SW team organization (2)

Project manager, team lead, team members

- Eng. management (often less technical, mostly business)
  - Project manager
    - Tech lead or chief architect or Lead programmer (<u>can be several)</u>
      - SW team members (coming from same or different organizations)
- Optional: external vendors, subcontractors
- Can have personnel manager in case of matrix management (manages people salary, vacations, promotions etc.)
- In case of global teams: some roles can be repeated locally (e.g. team lead)

Important

Lead programmer or Team lead or Project Manager

- Usually: *lead programmer* denotes primarily technical role, while *team lead* also includes some managerial tasks, and *project manager* is focused mostly on resource management and scheduling

- Lead programmer Team lead Project Manager

- Definition for lead programmer from Wikipedia
  - http://en.wikipedia.org/wiki/Lead_programmer

Team lead roles

Important

- From: http://www.npd-solutions.com/leader.html (Great summary)
  - Provide team leadership and coaching
  - Focus the team on the tasks at hand or the internal and external customer requirements
  - Coordinate team logistics
  - Communicate team status, task accomplishment, and direction

- Example job advertisements
  - https://www.livecareer.com/resumes/samples/leader/software-development-team-leader
  - https://teksavvy.com/Media/Default/Job%20Descriptions/jd-Software%20Development%20Team%20Lead-en-03.12.13.pdf

Important

Team CTO/tech lead role

- Also: lead programmer, lead/SW architect

  Definitions, roles, responsibilities:
  - Establishes a technical vision with the development team
  - Leads SW Architecture
  - Works with the team on development
  - Resolves technical issues and trade-offs
  - Gives technical advice to team lead
- Resources
  - https://www.thekua.com/atwork/2014/11/the-definition-of-a-tech-lead/
  - http://devterra.com/downloads/careers/Devterra-Careers_Technical-Lead.pdf
  - https://en.wikipedia.org/wiki/Lead_programmer
  - https://en.wikipedia.org/wiki/Software_architect

Tech Leads – can be several in a team

- Larger SW teams can have several tech leads, each specializing in some sub-area
  - E.g. Lead Architect
- Our class:
  - Back end lead
    - Data organization, database, APIs for front end, overall system architecture…also leads its team
  - Front end lead
    - UI design (including "customers"), business and application logic…also leads its team
  - Back end and front end lead have to cooperate and establish good SW interface

Team github master

- "Owns" github organization
- Manages branches
- Ensures integrity of master branch
- Reviews githib postings (for comments etc.)
- Good best practices for github teams
  - https://stackify.com/managing-teams-github/
  - https://code.tutsplus.com/articles/team-collaboration-with-github--net-29876
  - https://medium.freecodecamp.org/follow-these-simple-rules-and-youll-become-a-git-and-github-master-e1045057468f
  - https://github.com/erlang/otp/wiki/writing-good-commit-messages

Globalization, outsourcing and teamwork

Important

Motivation

- Get talent you need in a very short time
- Access to local market with specific requirements
- Reduce cost (do not forget cost of maintenance, problem handling) – becoming less of as factor lately
- Simplify adherence to specific county legal, import and business requirements

Teamwork related positives:

- Meet wonderful new people from different cultures
- Make new personal, professional and business connections
- Travel to distant lands

Important

Globalization, outsourcing and teamwork

Significant Challenges:

- Complicated management (people, business, project, multiple management chains)
- Teams may have different objectives
- Loss of coordination
- Harder to resolve conflicts
- Cultural differences, diverse team members
- Impediments due to *time zone differences and lack of personal contact*

Important

Globalization, outsourcing and teamwork

- If you are part of global team:
  - Be very aware of the challenges specific to globalization
  - Work on establishing personal connection **early** – use phone, Skype, video, images for introduction, travel if possible
  - Create common goals
  - Be sensitive to cultural and diversity issues
  - Create process for conflict resolution

Important

Behavior and attitude are critical in conflict management

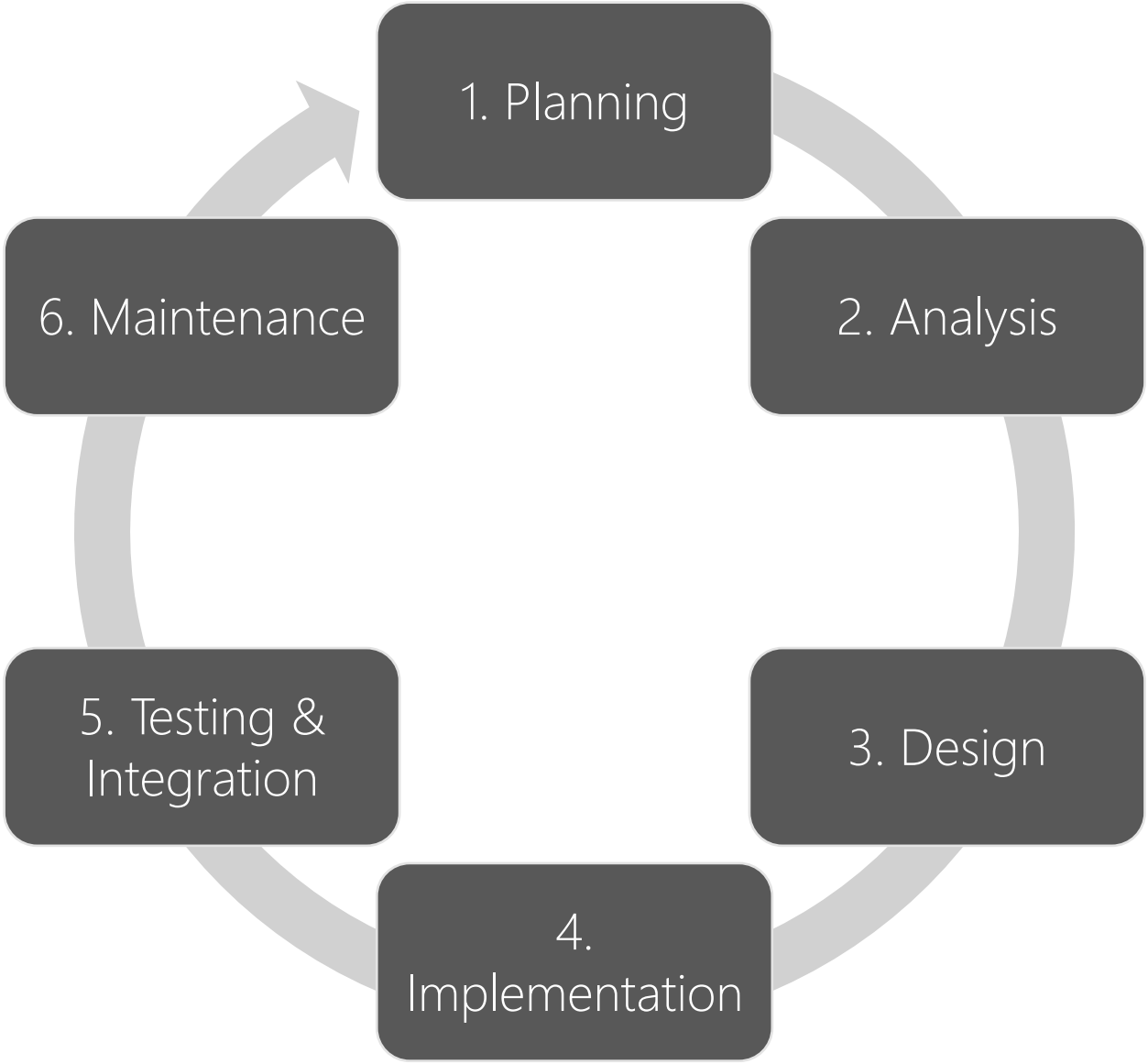
- Separate technical from personal issues
- Be courteous
- *Focus on problems, not personalities*
- *Fight for your ideas, but must ultimately think of what is best for the project, the team, the company*
- Do not send problematic e-mail, use meetings or phone
- For managers and team leads: encourage “out of box thinking” and balance it with the need to come to common plan and execute on time and budget and per specs

Teamwork issues: personal behavior and teamwork ethics are critical

Important

- Phone manners
  - http://ezinearticles.com/?20--Business-Telephone-Etiquette-Tips&id=246471
- E-mail manners
  - http://careerplanning.about.com/od/communication/a/email_etiquette.htm
- Meeting manners
  - http://www.chatgrispress.com/Articles/Business/11_Commandments_for_Bus_Meet_Etiquette.pdf
- Attend meetings on time or let people know if you can not
- Respect different personalities and cultures
- Dealing with conflict
- Assertiveness vs. aggressiveness
  - http://bookboon.com/blog/2012/12/whats-the-difference-between-an-assertive-and-aggressive-leader/
- Understand company culture and "Implicit behaviors and rules"
- *Global issues*: note: different countries have different culture and rules

Software Development Life Cycle (SDLC) / SE Process Lifecycle



SW Engineering Process Components

1. *Requirements and Specification*
2. *Design and Implementation*
3. *Validation*
4. *Maintenance and Evolution*

SW Engineering Process Components

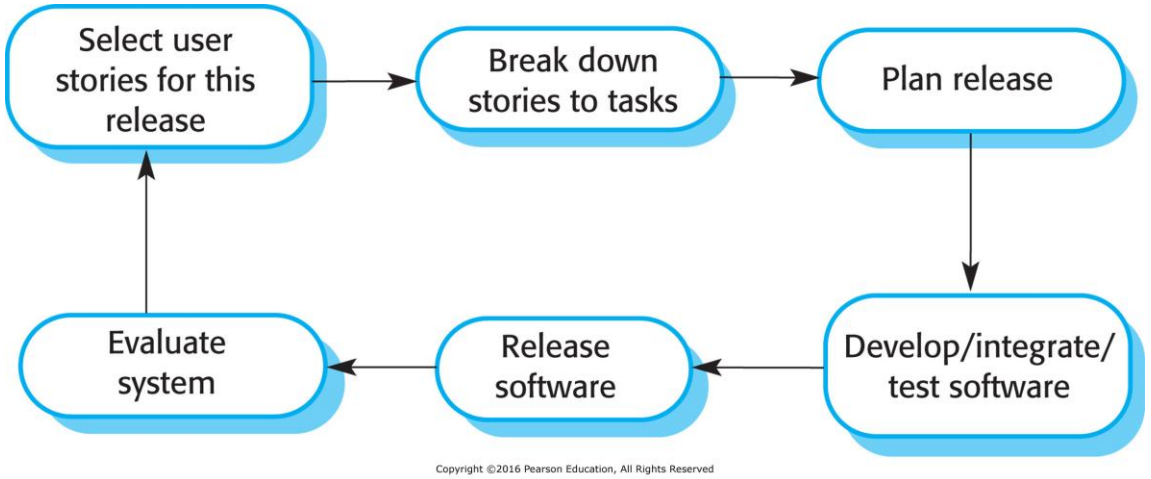
- There are several basic components present in all SW Engineering methodologies:
  1. *SW Requirements and Specification*: functionality, constraints, look and feel, GUI, performance
  2. *SW Design and Implementation*
  3. *SW Validation*: formal, test plans, QA
  4. *SW Maintenance and Evolution*: ongoing changes (and fixes) to meet customer needs

  These components are applied in different way and order (linear, iterative, spiral…) depending on chosen SW Engineering methodology or models (I.e. *waterfall, extreme, agile* etc.)

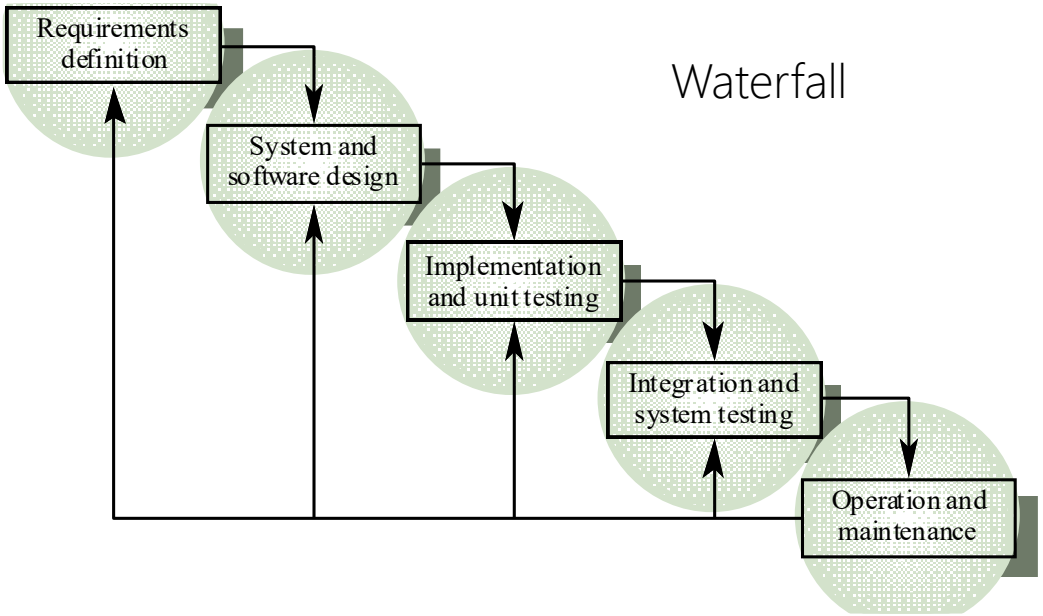
Methodologies

- Waterfall
- Evolutionary
- Iterative
- Incremental
- Open Source Development
- Extreme Programing
- User Centered Design (UCD)
- Continuous Development
- AGILE & SCRUM

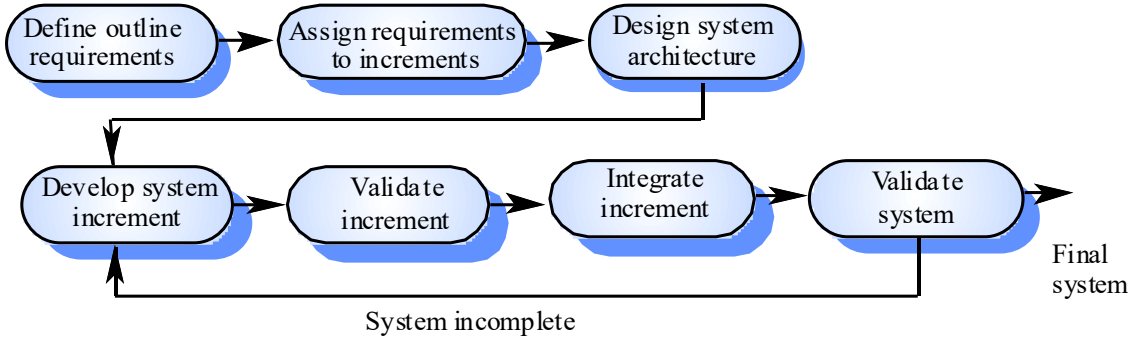
Extreme Programming



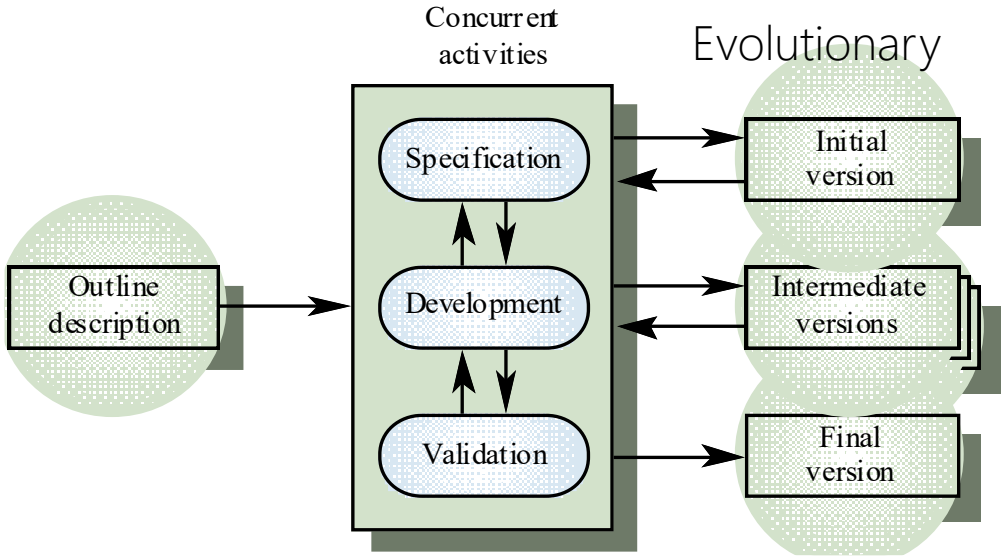
Waterfall



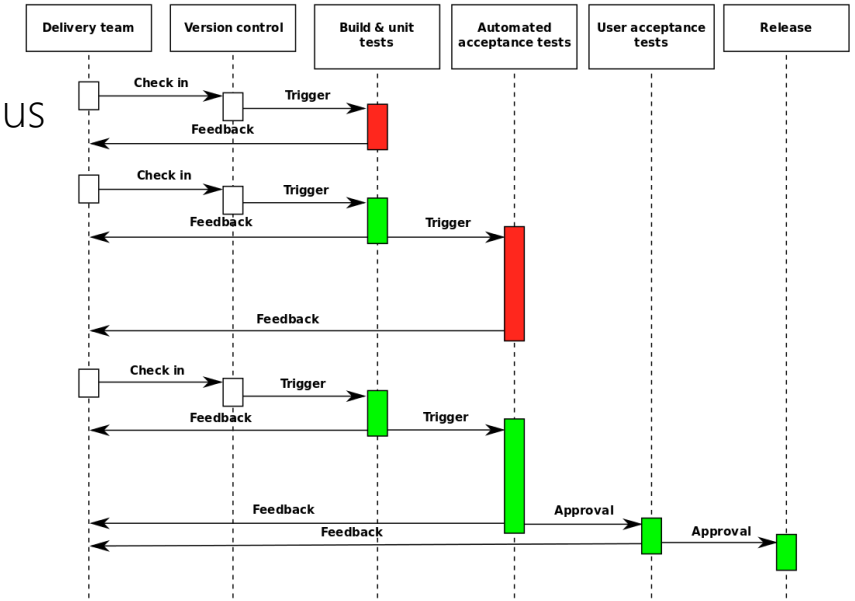
Incremental



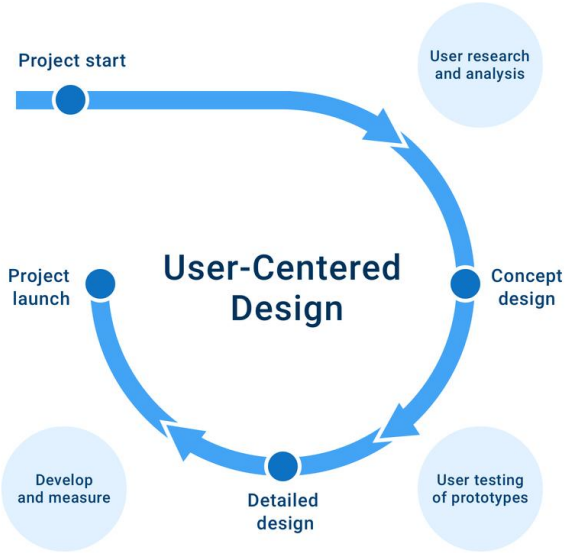
Evolutionary



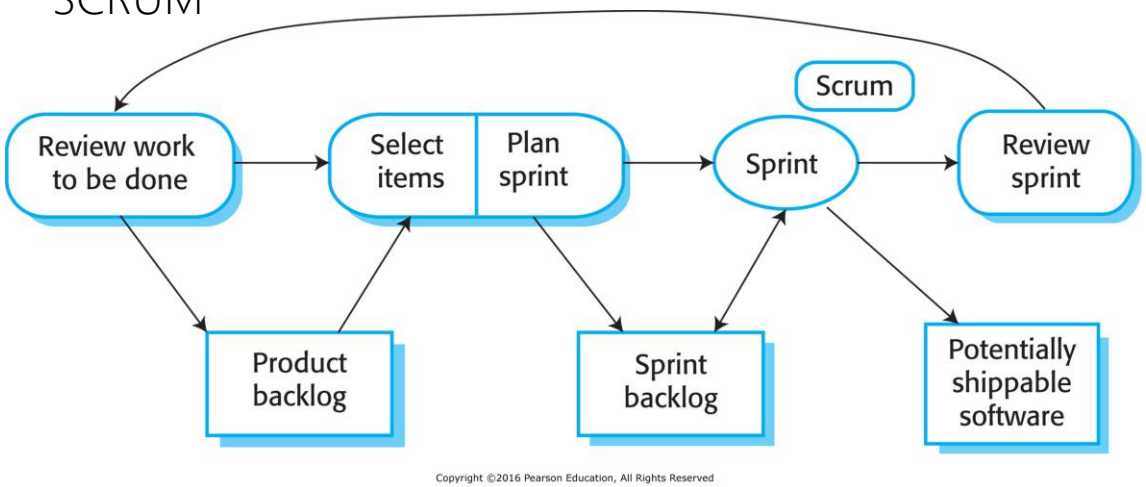
Continuous



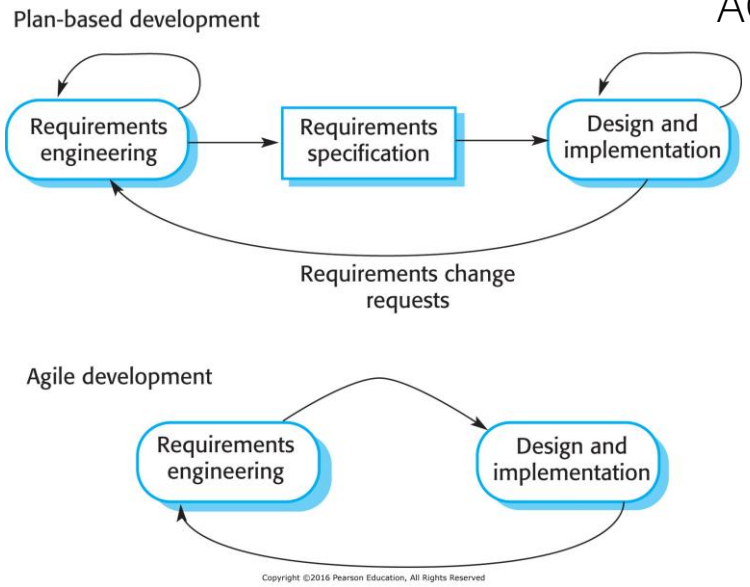
UCD



SCRUM



AGILE



Thank You